OVERVIEW **2**
COMPONENTS **2**
CONFIGURATION **3**
autoProvSetup.json **3**
url 3
validateCert 3
gateway 3
zone2addRouterInterfaceNamesToIfReq 3
logFileName 3
logFileSizeInBytes 3
logFileBackupCount 3
encryptAuth 3
logLevel 4
autoProvGroups.json **4**
provGroups 4
enabled 4
provGroup 4
deviceType 4
interfacePrefix 4
key 5
IP 5
Checking the JSON format files **5**
INSTALLATION **5**
Copy the script and config files to the central script folder **5**
Activate the ScriptRunner account 5
Activating autoProv.py **6**
Configuring Event Hooks 6
Sample log output **7**
Checking the log 7
Micetro Status pre-run 7
autoProvisioning.log 7
Micetro Status post run 8

Overview

autoProv.py is a Python based script written as an example to show how Micetro can be used for Network Automation

It does this by querying Micetro via the JSON-RPC/SOAP API (https://docs.bluecatnetworks.com/r/en-US/Micetro-User-Guide/SOAP-API-for-Micetro/25.2.0) for Ranges that have the Custom Property 'provision-grp', defined matching the router profile defined in the configuration file autoProvGroups.json The value of another custom property set at the Range level 'auto-provision' is then read and the following actions taken accordingly –

add → The network interface is added to the target router defined in 'provision-grp'
delete → The network interface is removed from the target router and the range is deleted from Micetro
none → No action is taken
provisioned → The network was previously successfully provisioned. The script will run some checks

This was written as an example and as such only works against VyOS routers using the VyOS api. However, the coded and config is structured such that this could easily be modified to work with other routers/switches/firewalls

I hope you find this useful…

Components

autoProv.py	Example automation script written in Python
autoProv.exe	x64 Windows binary/package created using pyinstaller https://pyinstaller.org/en/stable/
autoProvSetup.json	JSON format configuration file. Needs to be tailored for the target environment
autoProvGroups.json	JSON format file containing target router info Needs to be tailored for the target environment
INSTALL.pdf	This document
Automating_Network_Configuration_with_Micetro.pdf	A working Lab example

Configuration

The audit2syslog script references 1 configuration file

- autoProvSetup.json - Configuration data specific to the Micetro environment
- autoProvGroups.json - Target router configuration and AUTH data

autoProvSetup.json

Contains all the setup information unique to the Micetro install in JSON format.

Example config-

```
{
    "url": "https://10.50.0.30/mmws/api/command/",
    "validateCert" : false,
    "gateway": "firstIP",
    "zone2addRouterInterfaceNamesToIfReq": "routers.lab",
    "logFileName" : "autoProvisioning.log",
    "logFileSizeInBytes" : 10485760,
    "logFileBackupCount" : 5,
    "encryptAuth" : true,
    "fernetUser" : "Fernet",
    "logLevel" : 10
}
```

url

The location where the API can be found on the Micetro webserver
Eg:

- http://micetro.micetrolab.com/mmws/api/command
- https://micetro.micetrolab.com/mmws/api/command
- http://192.168.12.2/mmws/api/command

validateCert

If using https for connections to the API, do we validate the SSL certificate?

- true - If using a valid PKI issued certificate
- false - If using self-signed

gateway

At which end of the subnet should we create interface IP

- firstIP - first available IP after the Network Address
- lastIP - IP immediately before the broadcast address

zone2addRouterInterfaceNamesToIfReq

If set, the script will create forward and reverse DNS entries for the gateway IPs created. If this isn't required just leave this blank

logFileName

The name of the log file to create relative to the directory containing central (one level above scripts)

logFileSizeInBytes

Max size before rotation

logFileBackupCount

Number of copies to retain

encryptAuth

Do you want to encrypt your API credentials?

- true - A Fernet key (https://cryptography.io/en/latest/fernet/) will be stored in the fernetUser (specified below) account within Micetro, to be used to encrypt/decrypt data. Any clear text authentication data within autoProvGroups.json will be encrypted with this
- false - No encryption required

logLevel

The numeric value associated with the required log level. See table below

Level	Numeric value	What it means / When to use it
logging.**NOTSET**	**0**	When set on a logger, indicates that ancestor loggers are to be consulted to determine the effective level. If that still resolves to NOTSET, then all events are logged. When set on a handler, all events are handled
logging.**DEBUG**	**10**	Detailed information, typically only of interest to a developer trying to diagnose a problem
logging.**INFO**	**20**	Confirmation that things are working as expected
logging.**WARNING**	**30**	An indication that something unexpected happened, or that a problem might occur in the near future (e.g. 'disk space low'). The software is still working as expected
logging.**ERROR**	**40**	Due to a more serious problem, the software has not been able to perform some function
logging.**CRITICAL**	**50**	A serious error, indicating that the program itself may be unable to continue running

Eg. setting LogLevel: 20 will log info, warning and error events but not debug.
See https://docs.python.org/3/library/logging.html

autoProvGroups.json

Contains the config related to the switches/routers/firewalls to be provisioned
Example config -

```
{
  "provGroups": [
    {
      "enabled": true,
      "provGroup": "vyos01",
      "deviceType": "vyos",
      "interfacePrefix": "dum",
      "key": "DHFGdsds3434LLLTTsdshtbPPPfdfdq",
      "IP": "10.50.0.248"
    },
    {
      "enabled": true,
      "provGroup": "vyos02",
      "deviceType": "vyos",
      "interfacePrefix": "dum",
      "key": "youGdsds3434LLLTTsdshtbPFGuirt5",
      "IP": "10.50.0.247"
    }
  ]
}
```

provGroups

A JSON array containing the details for each individual device

enabled

true - If set to true the script will process this device
false - If set to false this device will be skipped

provGroup

The provisioning group name that this device belongs to. Needs to match the name set in the Micetro Custom Property 'provision-grp'

deviceType

The type of device to provision. This example script currently only supports 'vyos'

interfacePrefix

the script uses this as a prefix for the new interface name and then appends the Micetro internal range reference to it. This probably doesn't work for all devices but VyOS can support a dummy interface type which works quite well for lab use

key

The API key for the script to use to configure the device

IP

The Management IP address of the device

Checking the JSON format files

If you have python installed you can use json.tool as below-

```
python -m json.tool profiles.json
```

Alternatively, something like Visual Studio Code does a good job here

Installation

Copy the script and config files to the central script folder

Important

autoProvSetup.json and autoProvGroups.json need to be tailored for your environment

The autoProv.py script together with the JSON config files need to be copied to the scripts folder under the Miceto Central default directory

Alternatively you could use the autoProv.exe file which is a pyinstaller bundle containing all the required python components + script. This should work on Windows 2019 and above

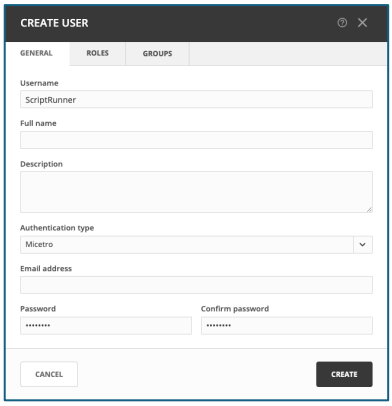
Eg.

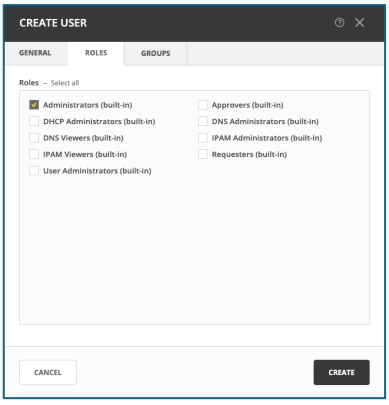
- default location for Windows: C:\ProgramData\Men and Mice\Central\scripts
- default location for Linux: /var/mmsuite/mmcentral/scripts

It's recommended that the JSON files are updated prior to transfer and checked to make sure the json is valid prior to transfer (visual studio code does a good job at this).

Activate the ScriptRunner account

As you would for a normal user create an account in Micetro called ScriptRunner and assign this to the administrator role.



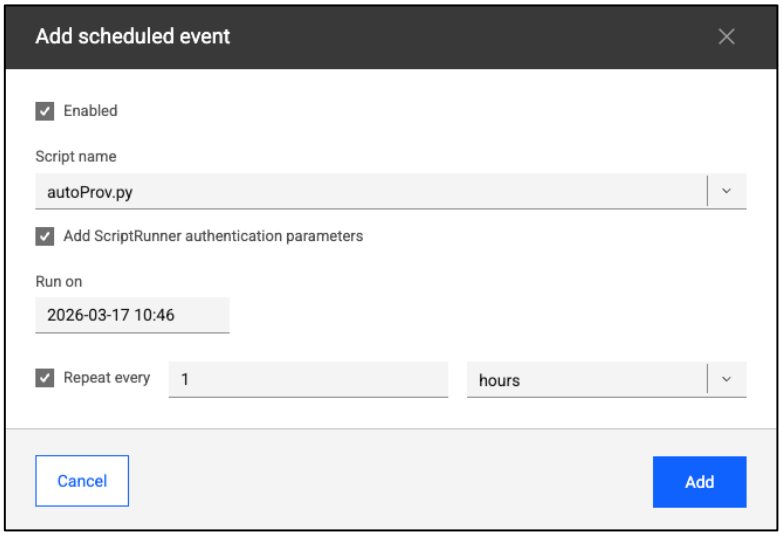


Although you add ScriptRunner in the normal way, this account is a special account in Micetro in that when used with scripts Micetro generates a unique password at each run.

Activating autoProv.py

Configuring Event Hooks

- Navigate to ADMIN | CONFIGURATION | EVENT HOOKS then click [+ ADD] under scheduled events



- Provided that autoProv.exe (or autoProv.py) has been placed in the scripts directory this should be available to select from the drop down.
- Tick Enabled
- Tick ‘Add ScriptRunner authentication Parameters’. Without this the script won’t be able to write back discovered data via the Micetro API
- Tick ‘Repeat Every’ and set to an hourly schedule (or whatever works for you)

Note
The default time after which Micetro will rotate the credentials for the ScriptRunner account is 300 seconds (5 mins). If autoProv takes longer to run than that then it is recommended that this be increased. To do this add the following to the end of the Micetro central preferences.cfg file-

```
<ScheduledScriptTimeout value="time_in_seconds" />
```

Sample log output

Checking the log

Note. The amount of detail in the logs will vary depending on the debug level set in autoProvSetup.json
The default location for the log is in the directory containing central. The logname and location can be adjusted relative to the central directory, again in autoProvSetup.json

Micetro Status pre-run

Range	Type	Utilization	auto-provision	provision-grp	Title	Router name	Gateway	Interface ID	Interface name	Interface description
10.222.0.0/16	Container	0%	none		vyos api demo subnets					
10.222.0.0/24	Range	1%	provisioned	vyos01	Network Auto Provisioning	vrapi01	10.222.0.1	149	dum3527	added by autoProv.py
10.222.9.0/24	Range	1%	delete	vyos01	Network Auto Provisioning	vrapi01	10.222.9.1	158	dum3533	added by autoProv.py
10.222.10.0/24	Range	0%	add	vyos02	vyos subnets					
10.222.11.0/24	Range	0%	add	vyos02	vyos subnets					
10.222.12.0/24	Range	0%	add	vyos02	vyos subnets					
10.222.13.0/24	Range	0%	add	vyos02	vyos subnets					
10.222.14.0/24	Range	0%	add	vyos02	vyos subnets					
10.222.1.0/24	Range	2%	provisioned	vyos01	Network Auto Provisioning	vrapi01	10.222.1.1	150	dum3524	added by autoProv.py
10.222.2.0/24	Range	1%	provisioned	vyos01	Network Auto Provisioning	vrapi01	10.222.2.1	151	dum3526	added by autoProv.py
10.222.3.0/24	Range	1%	provisioned	vyos01	Network Auto Provisioning	vrapi01	10.222.3.1	152	dum3525	added by autoProv.py
10.222.4.0/24	Range	1%	provisioned	vyos01	Network Auto Provisioning	vrapi01	10.222.4.1	153	dum3528	added by autoProv.py
10.222.5.0/24	Range	1%	delete	vyos01	Network Auto Provisioning	vrapi01	10.222.5.1	154	dum3529	added by autoProv.py
10.222.6.0/24	Range	1%	delete	vyos01	Network Auto Provisioning	vrapi01	10.222.6.1	155	dum3531	added by autoProv.py
10.222.7.0/24	Range	1%	delete	vyos01	Network Auto Provisioning	vrapi01	10.222.7.1	156	dum3530	added by autoProv.py
10.222.8.0/24	Range	1%	delete	vyos01	Network Auto Provisioning	vrapi01	10.222.8.1	157	dum3532	added by autoProv.py

autoProvisioning.log

With explanatory annotation ←

```
2026-03-17 11:46:52,076 - Starting Auto Provisioning v 0.0.1
2026-03-17 11:46:52,077 - Starting autoProvSetup.json format check v 0.0.1
2026-03-17 11:46:52,077 - Connecting to https://10.50.0.30/mmws/api/command/
2026-03-17 11:46:52,338 - Got response <Response [200]> from Micetro API
2026-03-17 11:46:52,339 - API test returned {'result': {'session': 'XZPtlT9xwO6w0QSxXT76sae48mS_9DfJGIKgPrSnUx'}}
2026-03-17 11:46:52,339 - creds ScriptRunner xxxxxxxxxxx
2026-03-17 11:46:52,339 - Finished autoProvSetup.json check
2026-03-17 11:46:52,344 - Parsing autoProvGroups.json
2026-03-17 11:46:52,354 - Checking to see if we have an encryption key stored in the Description Field of Fernet
2026-03-17 11:46:52,357 - Retrieving key from Fernet user description field
2026-03-17 11:46:52,357 - Fernet encryption key found
2026-03-17 11:46:52,361 - Processing Provisioning Group Profile: vyos01 ← Starting to process provision-grp vyos01
2026-03-17 11:46:52,361 - Looking for Networks related to deviceGroup vyos01
2026-03-17 11:46:52,368 - Setting up connection to vyos router: 10.50.0.248
2026-03-17 11:46:52,369 - Processing Micetro Range Reference ranges/3527 ← checking active network 10.222.0.0/24
2026-03-17 11:46:52,369 - Network 10.222.0.0/24 to vyos01 using gateway IP 10.222.0.1 and interface name dum3527 should be
already active
2026-03-17 11:46:52,637 - vyos API returned: ['dum3527:', '<BROADCAST,NOARP,UP,LOWER_UP>', 'mtu', '1500', 'qdisc', 'noqueue',
'state', 'UNKNOWN', 'group', 'default', 'qlen', '1000', 'link/ether', 'ba:3f:41:4b:c5:be']
2026-03-17 11:46:52,637 - Processing Micetro Range Reference ranges/3524 ← checking active network 10.222.1.0/24
2026-03-17 11:46:52,637 - Network 10.222.1.0/24 to vyos01 using gateway IP 10.222.1.1 and interface name dum3524 should be
already active
2026-03-17 11:46:52,909 - vyos API returned: ['dum3524:', '<BROADCAST,NOARP,UP,LOWER_UP>', 'mtu', '1500', 'qdisc', 'noqueue',
'state', 'UNKNOWN', 'group', 'default', 'qlen', '1000', 'link/ether', 'e6:52:53:0c:e7:1a']
2026-03-17 11:46:52,909 - Processing Micetro Range Reference ranges/3526 ← checking active network 10.222.2.0/24
2026-03-17 11:46:52,909 - Network 10.222.2.0/24 to vyos01 using gateway IP 10.222.2.1 and interface name dum3526 should be
already active
2026-03-17 11:46:53,169 - vyos API returned: ['dum3526:', '<BROADCAST,NOARP,UP,LOWER_UP>', 'mtu', '1500', 'qdisc', 'noqueue',
'state', 'UNKNOWN', 'group', 'default', 'qlen', '1000', 'link/ether', 'd2:3b:a6:b9:34:8c']
2026-03-17 11:46:53,169 - Processing Micetro Range Reference ranges/3525 ← checking active network 10.222.3.0/24
2026-03-17 11:46:53,169 - Network 10.222.3.0/24 to vyos01 using gateway IP 10.222.3.1 and interface name dum3525 should be
already active
2026-03-17 11:46:53,433 - vyos API returned: ['dum3525:', '<BROADCAST,NOARP,UP,LOWER_UP>', 'mtu', '1500', 'qdisc', 'noqueue',
'state', 'UNKNOWN', 'group', 'default', 'qlen', '1000', 'link/ether', 'c2:eb:43:7c:51:4b']
2026-03-17 11:46:53,434 - Processing Micetro Range Reference ranges/3528 ← checking active network 10.222.4.0/24
2026-03-17 11:46:53,434 - Network 10.222.4.0/24 to vyos01 using gateway IP 10.222.4.1 and interface name dum3528 should be
already active
2026-03-17 11:46:53,695 - vyos API returned: ['dum3528:', '<BROADCAST,NOARP,UP,LOWER_UP>', 'mtu', '1500', 'qdisc', 'noqueue',
'state', 'UNKNOWN', 'group', 'default', 'qlen', '1000', 'link/ether', '36:fe:10:78:8e:d9']
2026-03-17 11:46:53,695 - Processing Micetro Range Reference ranges/3529 ← deleting active network 10.222.5.0/24

2026-03-17 11:46:53,695 - Deleting Network 10.222.5.0/24 from vyos01 using gateway IP 10.222.5.1 and interface name dum3529
2026-03-17 11:46:54,834 - vyos API returned:
2026-03-17 11:46:54,834 - Checking and deleting any records associated with 10.222.5.1
2026-03-17 11:46:54,861 - Deleting IPAM object ref: ipamRecords/44429
2026-03-17 11:46:54,934 - Deleting object: <Response [204]>
```

```
2026-03-17 11:46:54,934 - deleting Network 10.222.5.0/24 from Micetro
2026-03-17 11:46:54,948 - Deleting object: <Response [204]>
2026-03-17 11:46:54,948 - Processing Micetro Range Reference ranges/3531 ← deleting active network 10.222.6.0/24
2026-03-17 11:46:54,948 - Deleting Network 10.222.6.0/24 from vyos01 using gateway IP 10.222.6.1 and interface name dum3531
2026-03-17 11:46:55,782 - vyos API returned:
2026-03-17 11:46:55,782 - Checking and deleting any records associated with 10.222.6.1
2026-03-17 11:46:55,794 - Deleting IPAM object ref: ipamRecords/44430
2026-03-17 11:46:55,868 - Deleting object: <Response [204]>
2026-03-17 11:46:55,868 - deleting Network 10.222.6.0/24 from Micetro
2026-03-17 11:46:55,875 - Deleting object: <Response [204]>
2026-03-17 11:46:55,875 - Processing Micetro Range Reference ranges/3530 ← deleting active network 10.222.7.0/24
2026-03-17 11:46:55,875 - Deleting Network 10.222.7.0/24 from vyos01 using gateway IP 10.222.7.1 and interface name dum3530
2026-03-17 11:46:56,627 - vyos API returned:
2026-03-17 11:46:56,627 - Checking and deleting any records associated with 10.222.7.1
2026-03-17 11:46:56,639 - Deleting IPAM object ref: ipamRecords/44419
2026-03-17 11:46:56,706 - Deleting object: <Response [204]>
2026-03-17 11:46:56,707 - deleting Network 10.222.7.0/24 from Micetro
2026-03-17 11:46:56,717 - Deleting object: <Response [204]>
2026-03-17 11:46:56,717 - Processing Micetro Range Reference ranges/3532 ← deleting active network 10.222.8.0/24
2026-03-17 11:46:56,717 - Deleting Network 10.222.8.0/24 from vyos01 using gateway IP 10.222.8.1 and interface name dum3532
2026-03-17 11:46:57,432 - vyos API returned:
2026-03-17 11:46:57,432 - Checking and deleting any records associated with 10.222.8.1
2026-03-17 11:46:57,443 - Deleting IPAM object ref: ipamRecords/44410
2026-03-17 11:46:57,515 - Deleting object: <Response [204]>
2026-03-17 11:46:57,515 - deleting Network 10.222.8.0/24 from Micetro
2026-03-17 11:46:57,524 - Deleting object: <Response [204]>
2026-03-17 11:46:57,525 - Processing Micetro Range Reference ranges/3533 ← deleting active network 10.222.9.0/24
2026-03-17 11:46:57,525 - Deleting Network 10.222.9.0/24 from vyos01 using gateway IP 10.222.9.1 and interface name dum3533
2026-03-17 11:46:58,219 - vyos API returned:
2026-03-17 11:46:58,219 - Checking and deleting any records associated with 10.222.9.1
2026-03-17 11:46:58,229 - Deleting IPAM object ref: ipamRecords/44431
2026-03-17 11:46:58,301 - Deleting object: <Response [204]>
2026-03-17 11:46:58,301 - deleting Network 10.222.9.0/24 from Micetro
2026-03-17 11:46:58,311 - Deleting object: <Response [204]>
2026-03-17 11:46:58,312 - Saving Changes to vyos host vyos01 IP 10.50.0.248
2026-03-17 11:46:58,312 - Processing Provisioning Group Profile: vyos02 ← Starting to process provision-grp vyos02
2026-03-17 11:46:58,312 - Looking for Networks related to deviceGroup vyos02
2026-03-17 11:46:58,319 - Setting up connection to vyos router: 10.50.0.247
2026-03-17 11:46:58,320 - Processing Micetro Range Reference ranges/3535 ← adding new network 10.222.10.0/24
2026-03-17 11:46:58,320 - Adding Network 10.222.10.0/24 to vyos02 using gateway IP 10.222.10.1 and interface name dum3535
2026-03-17 11:46:59,505 - vyos API returned: ApiResponse(status=200)
2026-03-17 11:46:59,587 - Created DNS Entry dum3535 for 10.222.10.1
2026-03-17 11:46:59,587 - Processing Micetro Range Reference ranges/3538 ← adding new network 10.222.11.0/24
2026-03-17 11:46:59,587 - Adding Network 10.222.11.0/24 to vyos02 using gateway IP 10.222.11.1 and interface name dum3538
2026-03-17 11:47:00,772 - vyos API returned: ApiResponse(status=200)
2026-03-17 11:47:00,849 - Created DNS Entry dum3538 for 10.222.11.1
2026-03-17 11:47:00,849 - Processing Micetro Range Reference ranges/3534 ← adding new network 10.222.12.0/24
2026-03-17 11:47:00,849 - Adding Network 10.222.12.0/24 to vyos02 using gateway IP 10.222.12.1 and interface name dum3534
2026-03-17 11:47:02,055 - vyos API returned: ApiResponse(status=200)
2026-03-17 11:47:02,134 - Created DNS Entry dum3534 for 10.222.12.1
2026-03-17 11:47:02,135 - Processing Micetro Range Reference ranges/3536 ← adding new network 10.222.13.0/24
2026-03-17 11:47:02,135 - Adding Network 10.222.13.0/24 to vyos02 using gateway IP 10.222.13.1 and interface name dum3536
2026-03-17 11:47:03,358 - vyos API returned: ApiResponse(status=200)
2026-03-17 11:47:03,439 - Created DNS Entry dum3536 for 10.222.13.1
2026-03-17 11:47:03,440 - Processing Micetro Range Reference ranges/3537 ← adding new network 10.222.14.0/24
2026-03-17 11:47:03,440 - Adding Network 10.222.14.0/24 to vyos02 using gateway IP 10.222.14.1 and interface name dum3537
2026-03-17 11:47:04,674 - vyos API returned: ApiResponse(status=200)
2026-03-17 11:47:04,742 - Created DNS Entry dum3537 for 10.222.14.1
2026-03-17 11:47:04,743 - Saving Changes to vyos host vyos02 IP 10.50.0.247
2026-03-17 11:47:04,743 - Finished
```

Micetro Status post run

Range	Type	Utilization	auto-provisic	provision-grp	Title	Router name	Gateway	Interface ID	Interface nar	Interface description
10.222.0.0/16	Container	0%	none		vyos api demo subnets					
10.222.0.0/24	Range	1%	provisioned	vyos01	Network Auto Provisioning	vrapi01	10.222.0.1	149	dum3527	added by autoProv.py
10.222.10.0/24	Range	1%	provisioned	vyos02	vyos subnets	vrapi02	10.222.10.1	32	dum3535	added by autoProv.py
10.222.11.0/24	Range	1%	provisioned	vyos02	vyos subnets	vrapi02	10.222.11.1	33	dum3538	added by autoProv.py
10.222.12.0/24	Range	1%	provisioned	vyos02	vyos subnets	vrapi02	10.222.12.1	34	dum3534	added by autoProv.py
10.222.13.0/24	Range	1%	provisioned	vyos02	vyos subnets	vrapi02	10.222.13.1	35	dum3536	added by autoProv.py
10.222.14.0/24	Range	1%	provisioned	vyos02	vyos subnets	vrapi02	10.222.14.1	36	dum3537	added by autoProv.py
10.222.1.0/24	Range	2%	provisioned	vyos01	Network Auto Provisioning	vrapi01	10.222.1.1	150	dum3524	added by autoProv.py
10.222.2.0/24	Range	1%	provisioned	vyos01	Network Auto Provisioning	vrapi01	10.222.2.1	151	dum3526	added by autoProv.py
10.222.3.0/24	Range	1%	provisioned	vyos01	Network Auto Provisioning	vrapi01	10.222.3.1	152	dum3525	added by autoProv.py
10.222.4.0/24	Range	1%	provisioned	vyos01	Network Auto Provisioning	vrapi01	10.222.4.1	153	dum3528	added by autoProv.py